

8/9/2025

TP3

Desenvolvimento de Serviços com SpringBoot

Professor(a): Flávio da Silva Neves

LINK GITHUB

https://github.com/faculdade-infnet/V-1-Spring_Boot/tree/main/TP3

1. GERENCIAMENTO DE DEPENDÊNCIAS

O gerenciamento de dependências do projeto foi realizado com Maven, facilitando o controle de versões e integração das bibliotecas necessárias. Isso garante que todas as dependências sejam resolvidas automaticamente.

Dependências escolhidas e justificativas

1. **spring-boot-starter-web**

Adiciona suporte para criação de aplicações web e APIs REST, incluindo o uso do Spring MVC e o servidor embutido Tomcat.

2. **spring-boot-devtools**

Melhora a produtividade no desenvolvimento, oferecendo recursos como hot reload e reinicialização automática da aplicação sempre que há mudanças no código.

3. **org.projectlombok:lombok**

Reduz a quantidade de código e facilita a manutenção com anotações que geram automaticamente getters, setters, construtores e métodos utilitários.

4. **spring-boot-starter-data-jpa**

Fornece integração com o JPA e Hibernate, permitindo trabalhar com persistência de dados de forma simplificada, usando repositórios e mapeamento objeto-relacional.

5. **com.h2database:h2**

Banco de dados em memória/arquivo, útil para testes e desenvolvimento. Permite executar a aplicação sem necessidade de instalar um banco externo, além de integrar facilmente com o DBeaver ou console web do H2.

6. **spring-boot-starter-test**

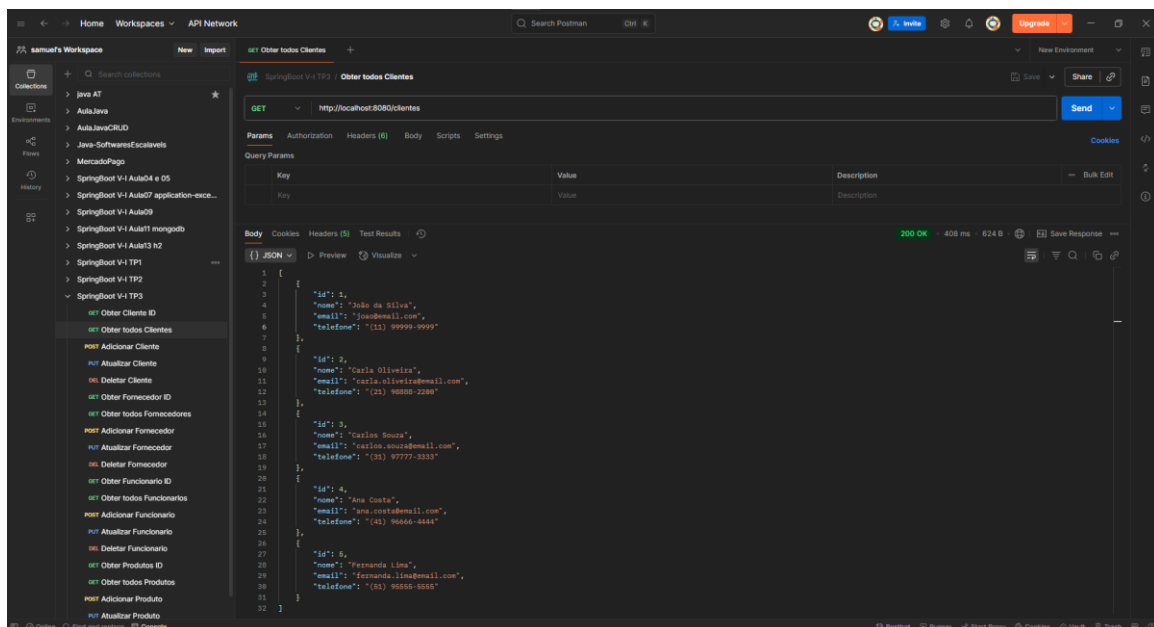
Fornece o conjunto de bibliotecas para testes (JUnit 5, Mockito, AssertJ, entre outros), permitindo implementar testes unitários e de integração de forma prática.

2. VALIDAÇÃO DOS CÓDIGOS DE STATUS HTTP DE ACORDO COM AS ESPECIFICAÇÕES REST

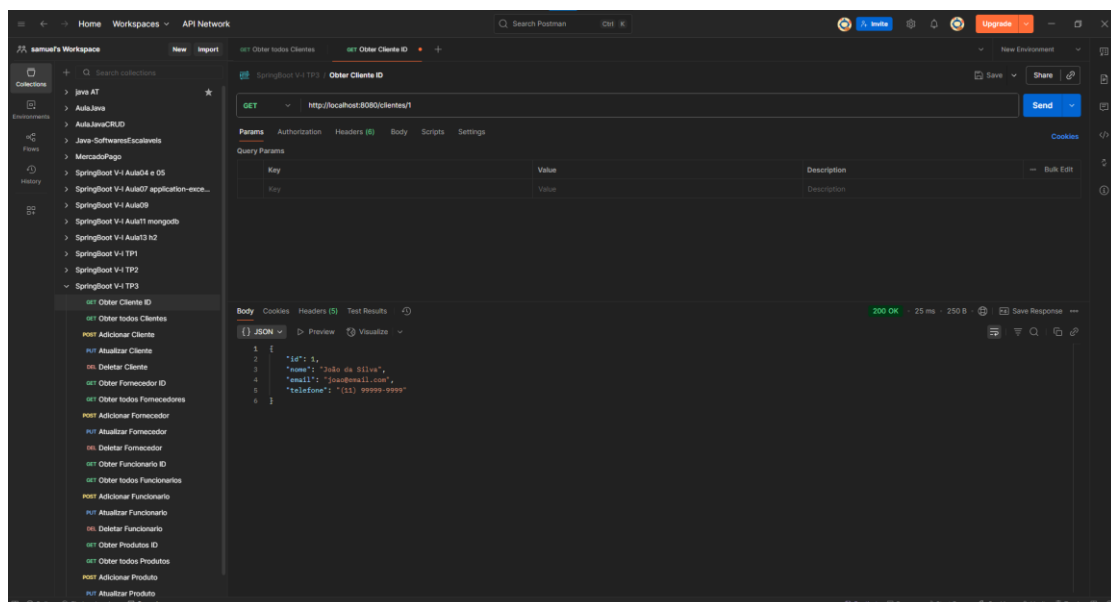
1. Clientes

Método HTTP	Endpoint	Código de Status	Justificativa
GET /clientes	Todos os clientes	200 OK	A requisição foi bem-sucedida e os dados foram retornados.
GET /clientes/{id}	Cliente por id	200 OK	Requisição bem-sucedida e o cliente foi encontrado.
POST /clientes	Criação de cliente	201 Created	Um novo recurso foi criado com sucesso.
PUT /clientes/{id}	Atualização de cliente	200 OK	Requisição bem-sucedida e o recurso atualizado foi retornado.
DELETE /clientes/{id}	Deletar cliente	204 No Content	Requisição bem-sucedida e não há conteúdo a retornar após a exclusão.

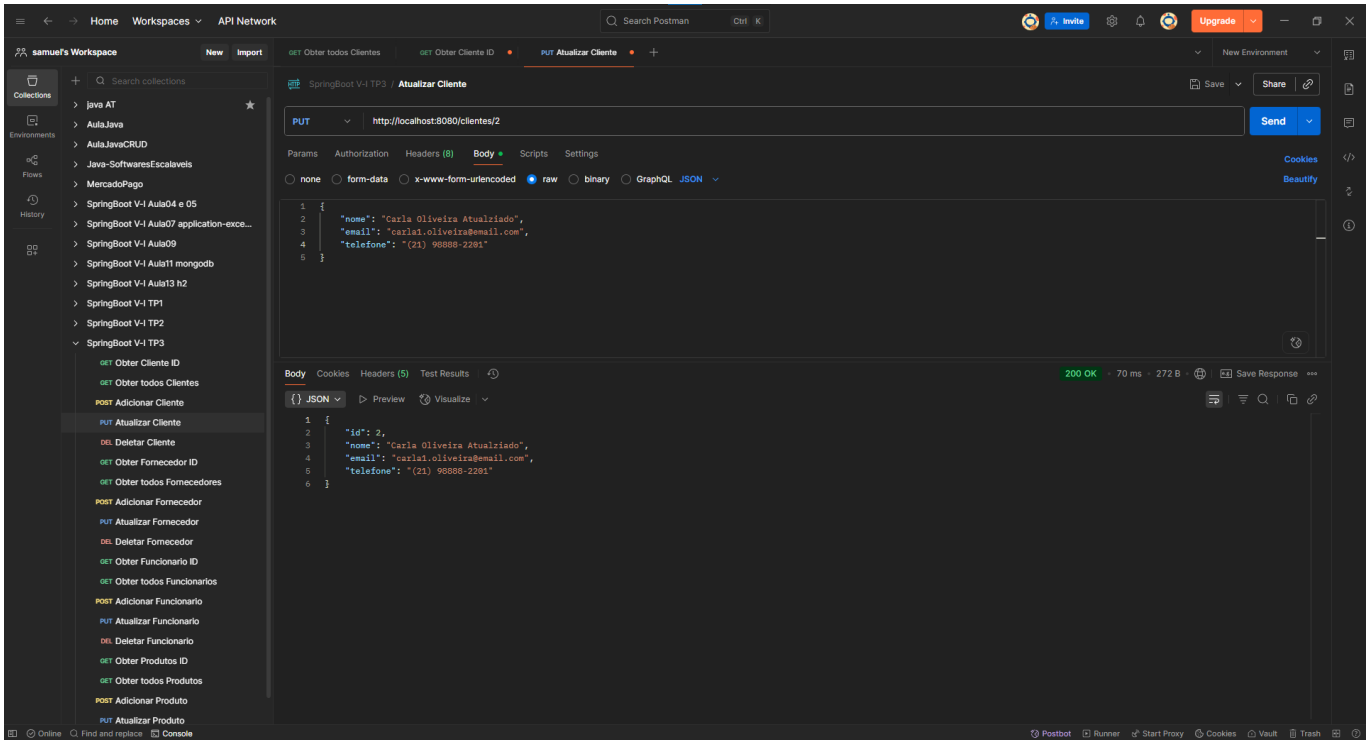
- Obtém Todos os Clientes



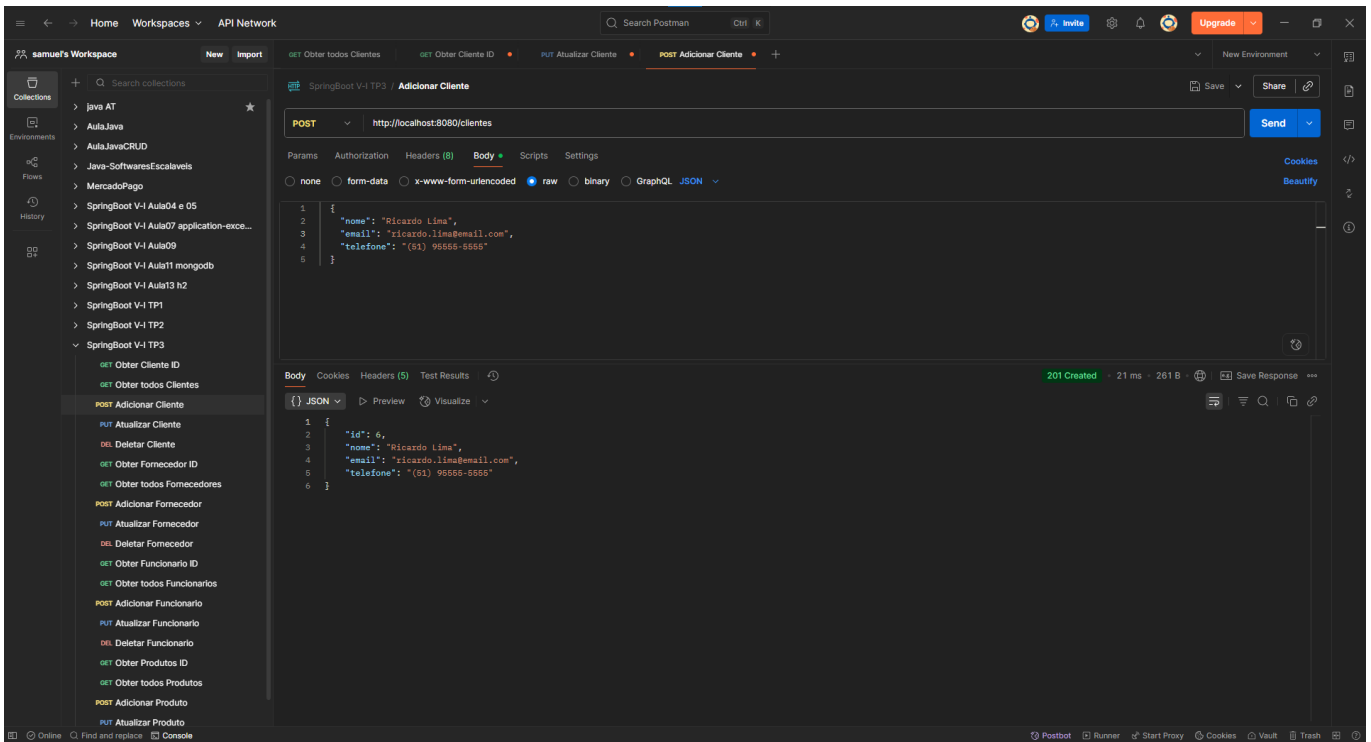
- Obtém um Cliente pelo ID



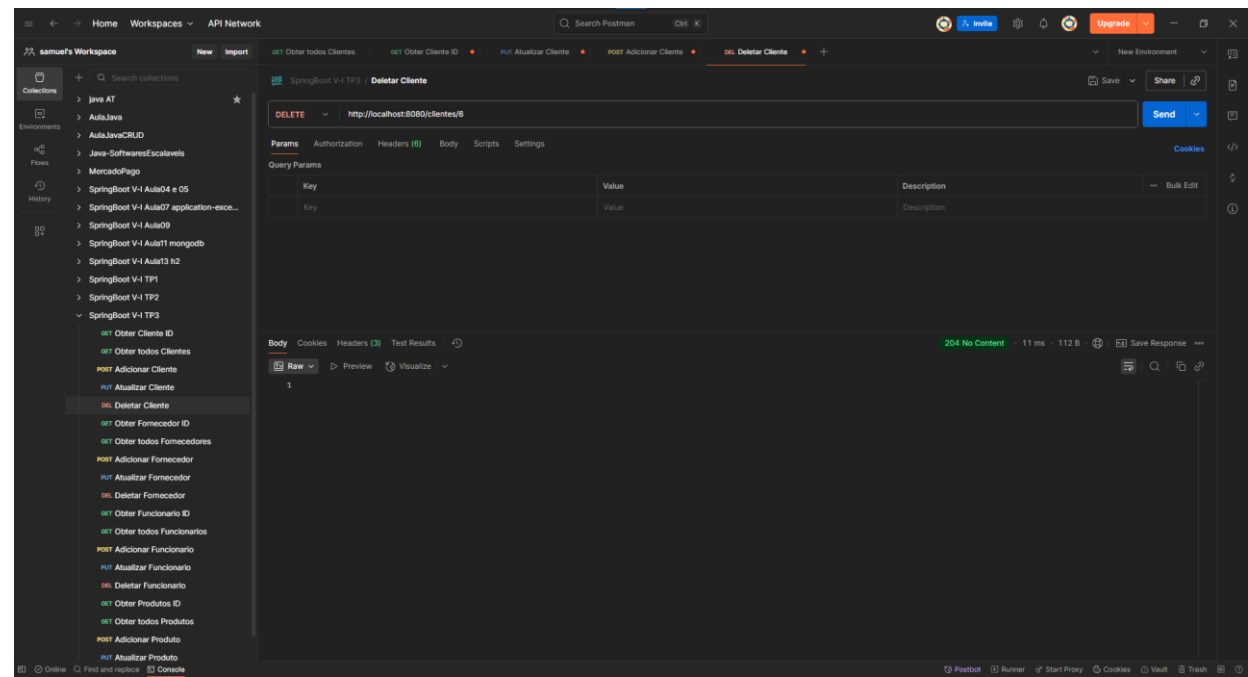
- Adiciona um novo Cliente



- Atualiza um Cliente pelo ID



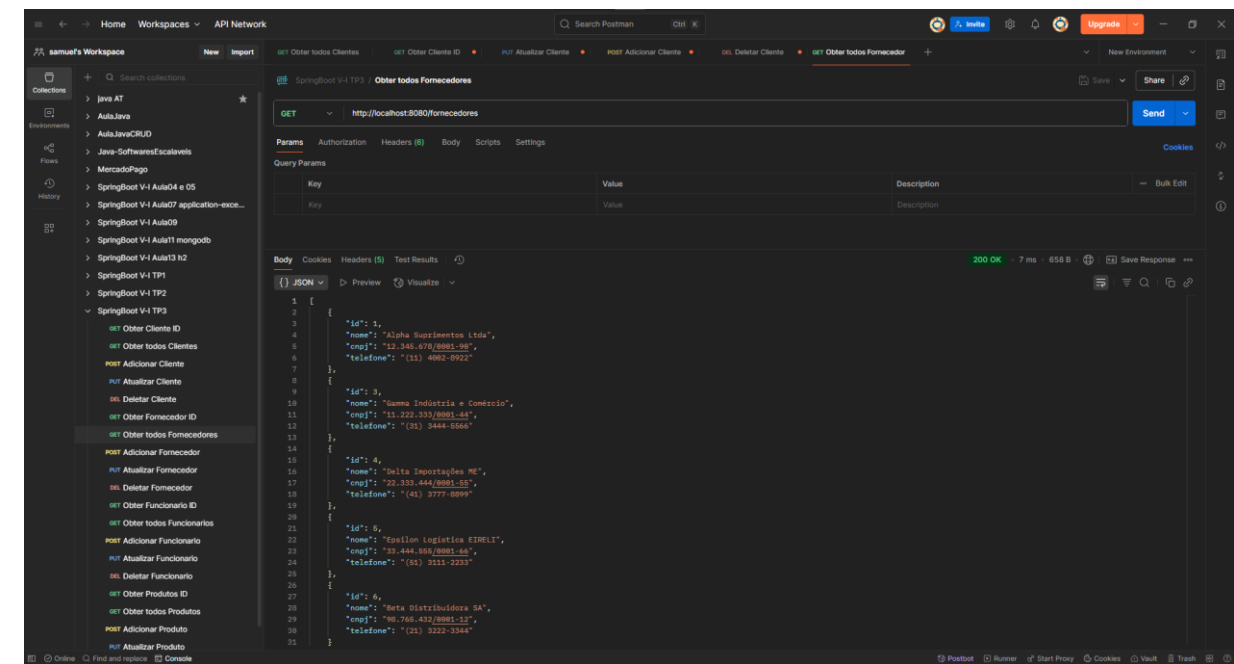
- Deleta um Cliente pelo ID



2. Fornecedores

Método HTTP	Endpoint	Código de Status	Justificativa
GET /fornecedores	Todos os fornecedores	200 OK	A requisição foi bem-sucedida e os dados foram retornados.
GET /fornecedores/{id}	Fornecedor por id	200 OK	Requisição bem-sucedida e o fornecedor foi encontrado.
POST /fornecedores	Criação de fornecedor	201 Created	Um novo recurso foi criado com sucesso.
PUT /fornecedores/{id}	Atualização de fornecedor	200 OK	Requisição bem-sucedida e o recurso atualizado foi retornado.
DELETE /fornecedores/{id}	Deletar fornecedor	204 No Content	Requisição bem-sucedida e não há conteúdo a retornar após a exclusão.

- Obtém Todos os Fornecedores



- **Obtém um Fornecedor pelo ID**

The screenshot shows the Postman interface with a workspace named 'samuel's Workspace'. The left sidebar lists various API collections, with 'SpringBoot V4 TP3' expanded. The main panel displays a GET request to 'http://localhost:8080/fornecedores/3'. The response is a JSON object with the following data:

```
{  "id": 3,  "nome": "Gama Indústria e Comércio",  "cnpj": "11.222.333/8881-44",  "telefone": "(31) 3444-5566"}
```

The status bar at the bottom indicates a '200 OK' response with a time of 8 ms and a size of 266 B.

- **Adiciona um novo Fornecedor**

The screenshot shows the Postman interface with a workspace named 'samuel's Workspace'. The left sidebar lists various API collections, with 'SpringBoot V4 TP3' expanded. The main panel displays a POST request to 'http://localhost:8080/fornecedores'. The request body is a JSON object with the following data:

```
{  "nome": "Bebidas Zonta",  "cnpj": "98.765.432/8881-88",  "telefone": "(21) 3222-3344"}
```

The response is a JSON object with the following data:

```
{  "id": 7,  "nome": "Bebidas Zonta",  "cnpj": "98.765.432/8881-88",  "telefone": "(21) 3222-3344"}
```

The status bar at the bottom indicates a '201 Created' response with a time of 5 ms and a size of 256 B.

- Atualiza um Fornecedor pelo ID

The screenshot shows the Postman interface with a workspace named 'samuel's Workspace'. The left sidebar lists various API collections, with 'SpringBoot V-4 TP3' expanded. The main panel displays a PUT request to 'http://localhost:8080/fornecedores/3'. The request body is a JSON object:

```
{  "nome": "Fornecedor Atualizado",  "cnpj": "11.222.333/0001-00",  "telefone": "(31) 3444-5588"}
```

. The response is a 200 OK status with a JSON body:

```
{  "id": 3,  "nome": "Fornecedor Atualizado",  "cnpj": "11.222.333/0001-00",  "telefone": "(31) 3444-5588"}
```

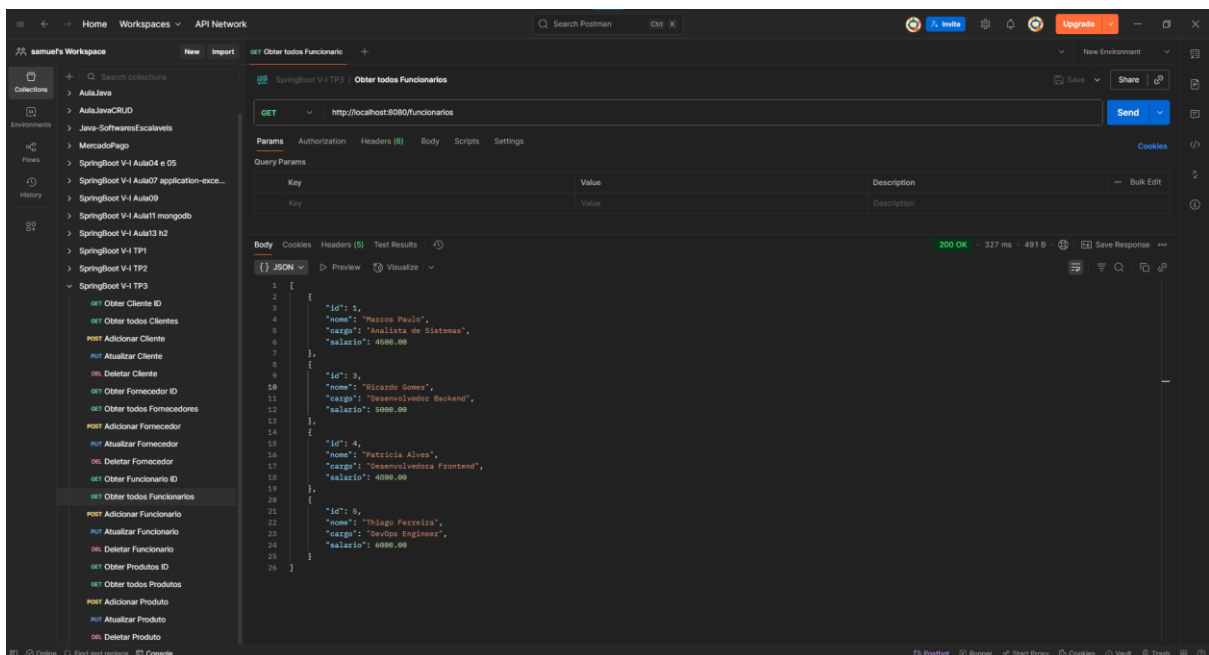
- Deleta um Fornecedor pelo ID

The screenshot shows the Postman interface with the same workspace. The main panel displays a DELETE request to 'http://localhost:8080/fornecedores/7'. The response is a 204 No Content status. The left sidebar shows the 'DEL Deletar Fornecedor' endpoint selected under the 'SpringBoot V-4 TP3' collection.

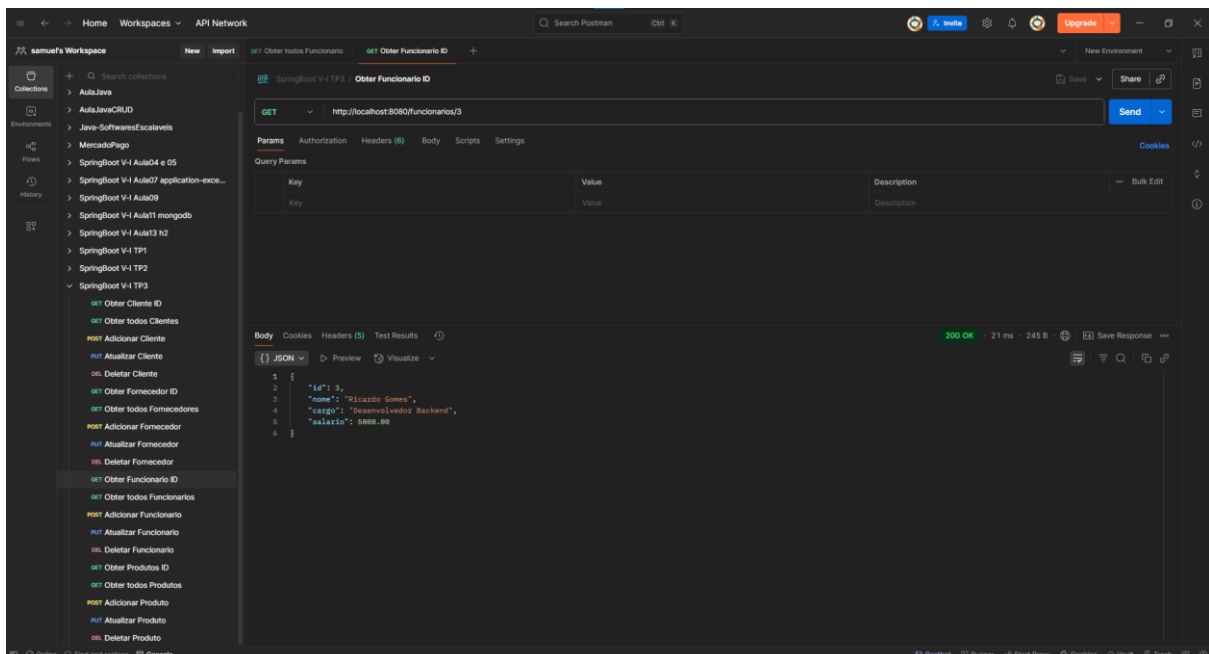
3. Funcionarios

Método HTTP	Endpoint	Código de Status	Justificativa
GET /funcionarios	Todos os funcionarios	200 OK	A requisição foi bem-sucedida e os dados foram retornados.
GET /funcionarios/{id}	Funcionario por id	200 OK	Requisição bem-sucedida e o funcionario foi encontrado.
POST /funcionarios	Criação de funcionario	201 Created	Um novo recurso foi criado com sucesso.
PUT /funcionarios/{id}	Atualização de funcionario	200 OK	Requisição bem-sucedida e o recurso atualizado foi retornado.
DELETE /funcionarios/{id}	Deletar funcionario	204 No Content	Requisição bem-sucedida e não há conteúdo a retornar após a exclusão.

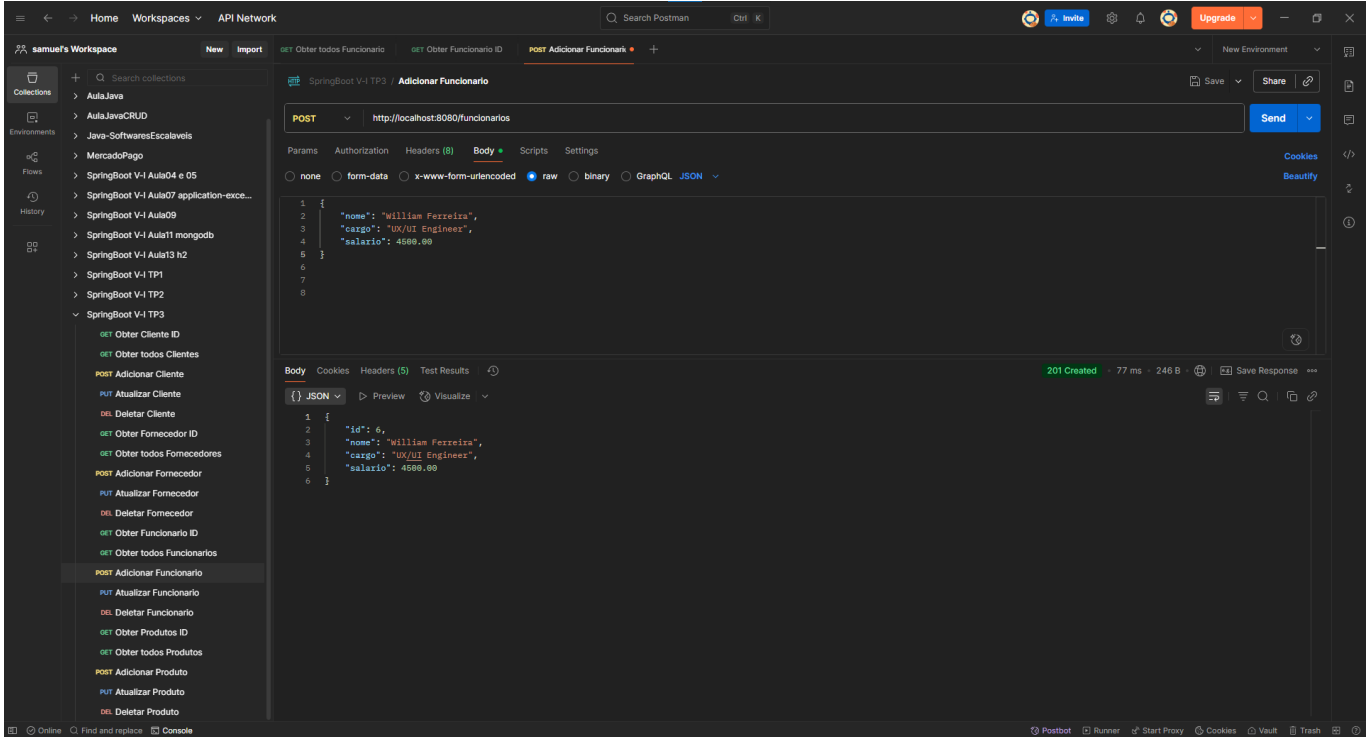
- **Obtém Todos os Funcionarios**



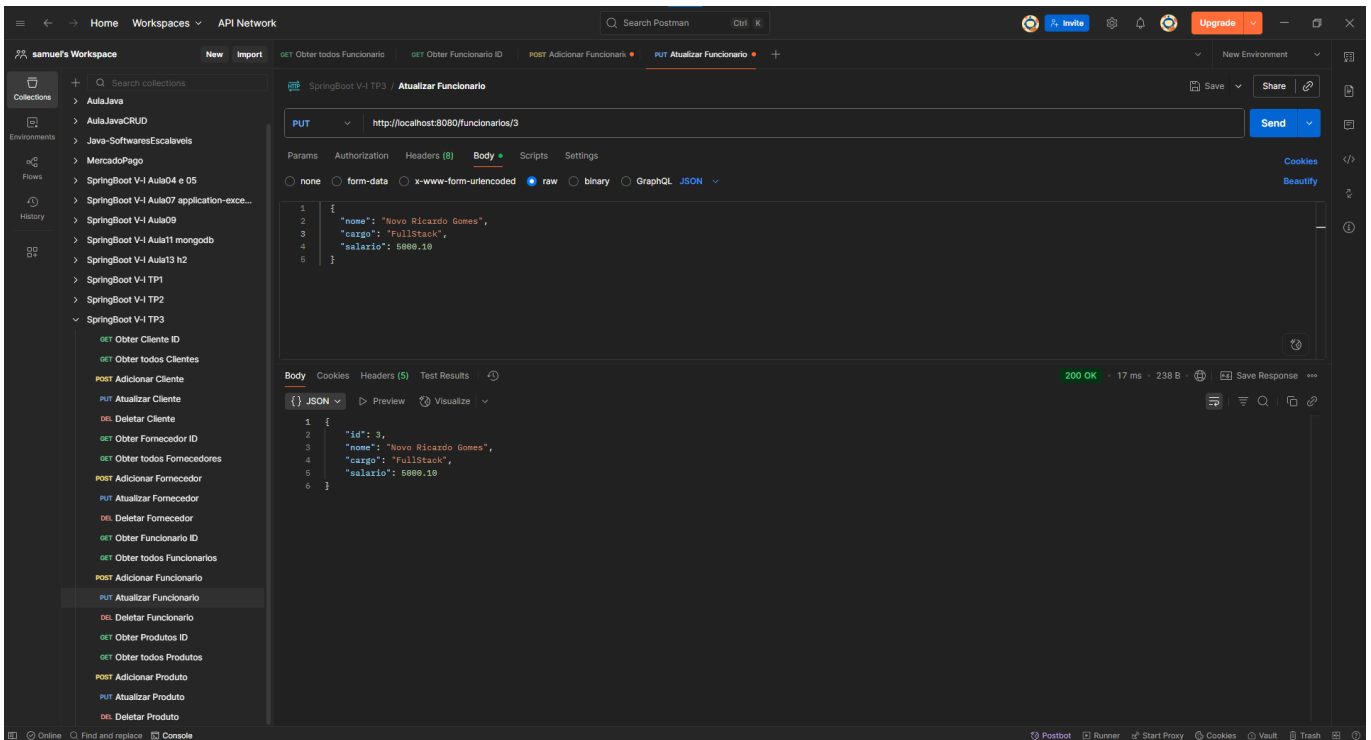
- **Obtém um Funcionario pelo ID**



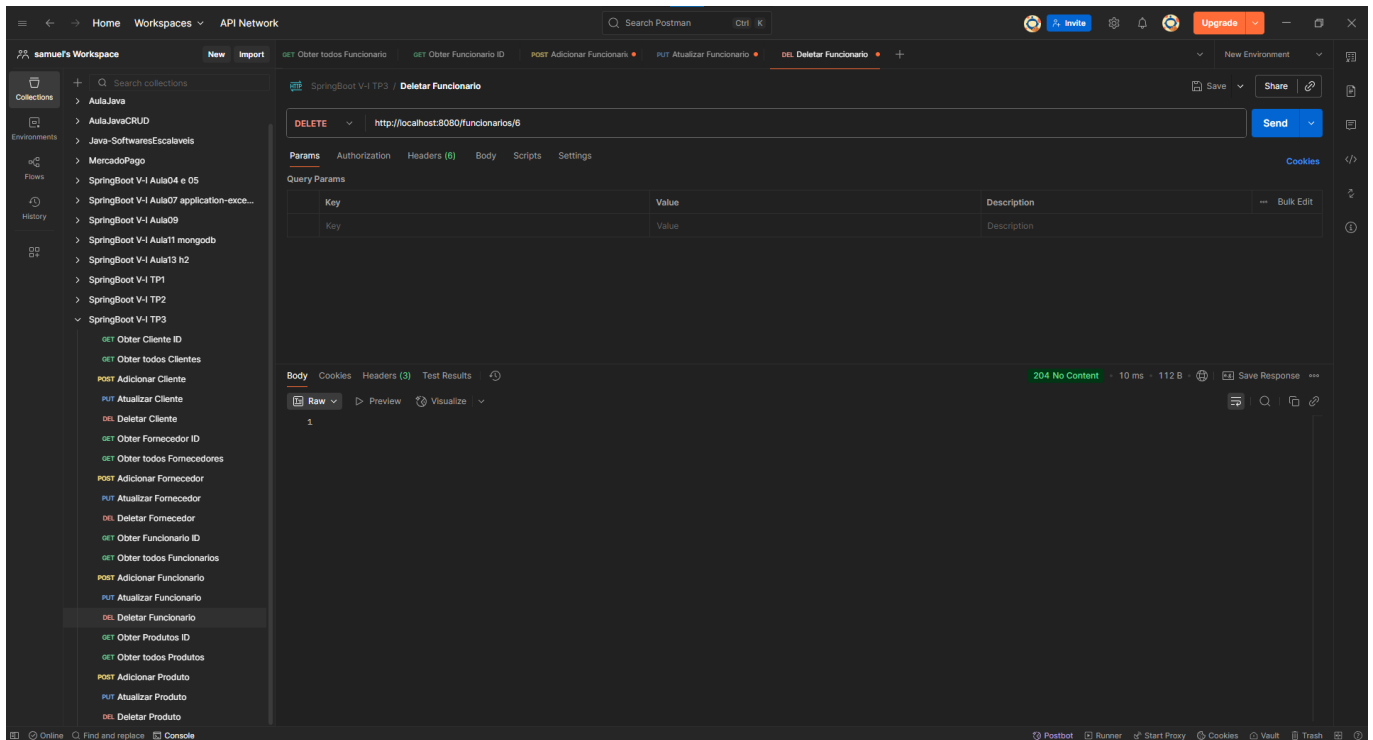
- Adiciona um novo Funcionario



- Atualiza um Funcionario pelo ID



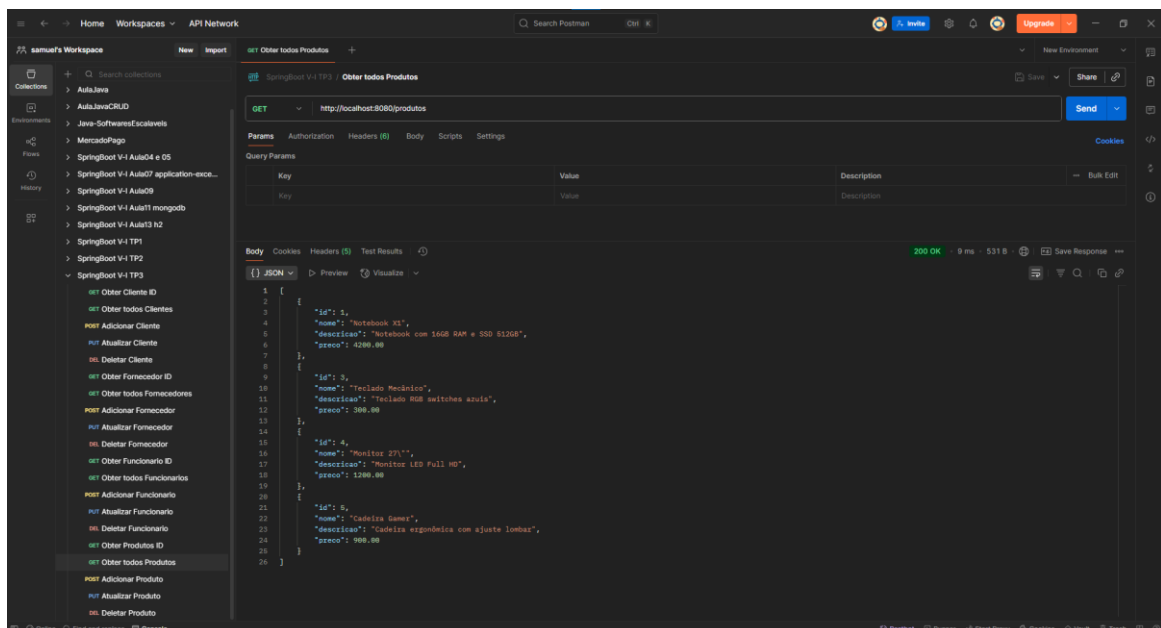
- Deleta um Funcionario pelo ID



4. Produtos

Método HTTP	Endpoint	Código de Status	Justificativa
GET /produtos	Todos os produtos	200 OK	A requisição foi bem-sucedida e os dados foram retornados.
GET /produtos/{id}	Produto por id	200 OK	Requisição bem-sucedida e o produto foi encontrado.
POST /produtos	Criação de cliente	201 Created	Um novo recurso foi criado com sucesso.
PUT /produtos/{id}	Atualização de cliente	200 OK	Requisição bem-sucedida e o recurso atualizado foi retornado.
DELETE /produtos/{id}	Deletar cliente	204 No Content	Requisição bem-sucedida e não há conteúdo a retornar após a exclusão.

- Obtém Todos os Produtos



- **Obtém um Produto pelo ID**

The screenshot shows the Postman interface with a workspace named 'samuel's Workspace'. The left sidebar lists various API endpoints under 'SpringBoot V-4 TP3', with 'GET Obter Produtos ID' selected. The main panel displays a GET request to 'http://localhost:8080/produtos/3'. The response is a 200 OK status with a 6 ms response time and 255 B of data. The response body is a JSON object:

```
{  "id": 3,  "nome": "Teclado Mecânico",  "descricao": "Teclado RGB switches azuis",  "preco": 399.99}
```

- **Adiciona um novo Produto**

The screenshot shows the Postman interface with the same workspace. The left sidebar lists various API endpoints under 'SpringBoot V-4 TP3', with 'POST Adicionar Produto' selected. The main panel displays a POST request to 'http://localhost:8080/produtos'. The request body is a JSON object:

```
{  "nome": "Pc Gamer",  "descricao": "PC ultima geração",  "preco": 9999.99}
```

The response is a 201 Created status with a 7 ms response time and 245 B of data. The response body is a JSON object:

```
{  "id": 6,  "nome": "Pc Gamer",  "descricao": "PC ultima geração",  "preco": 9999.99}
```

- Atualiza um Produto pelo ID

The screenshot shows the Postman interface with a workspace named 'samuel's Workspace'. The left sidebar lists various API collections, with 'SpringBoot V-4 TP3' expanded and 'PUT Atualizar Produto' selected. The main panel displays the configuration for a PUT request to 'http://localhost:8080/produtos/6'. The request body is a JSON object:

```
{  "nome": "Pc Gamer Novo",  "descricao": "Pc ultima geração novo",  "preco": 18888.88}
```

. The response tab shows a '200 OK' status with a JSON body:

```
{  "id": 6,  "nome": "Pc Gamer Novo",  "descricao": "Pc ultima geração novo",  "preco": 18888.88}
```

- Deleta um Produto pelo ID

The screenshot shows the Postman interface with the same workspace. The left sidebar shows 'SpringBoot V-4 TP3' expanded and 'DEL Deletar Produto' selected. The main panel displays the configuration for a DELETE request to 'http://localhost:8080/produtos/6'. The response tab shows a '204 No Content' status, indicating successful deletion.

3. DESENVOLVIMENTO DE SERVIÇOS REST COM PERSISTÊNCIA

1. Estrutura do Projeto

- Entity: representa a tabela no banco (ex: Cliente).
- Repository: interface JPA para CRUD automático.
- Service: lógica de negócio, chama o repository.
- Controller: expõe os endpoints REST.

2. Fluxo de uma Requisição para Cliente, Fornecedor, Funcionario e Produto.

- Cliente faz requisição HTTP.
- Controller recebe e chama Service.
- Service executa lógica e chama Repository.
- Repository acessa o banco e retorna os dados.
- Controller retorna ResponseEntity com status HTTP e dados.

3. Configuração do Banco (application.properties)

```
spring.application.name=TP3

server.port=8080
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:file:./tp3_db
spring.datasource.username=sa
spring.datasource.password=12345

spring.h2.console.enabled=true
# Endereço de acesso "http://localhost:8081/h2-database"
spring.h2.console.path=/h2-database

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

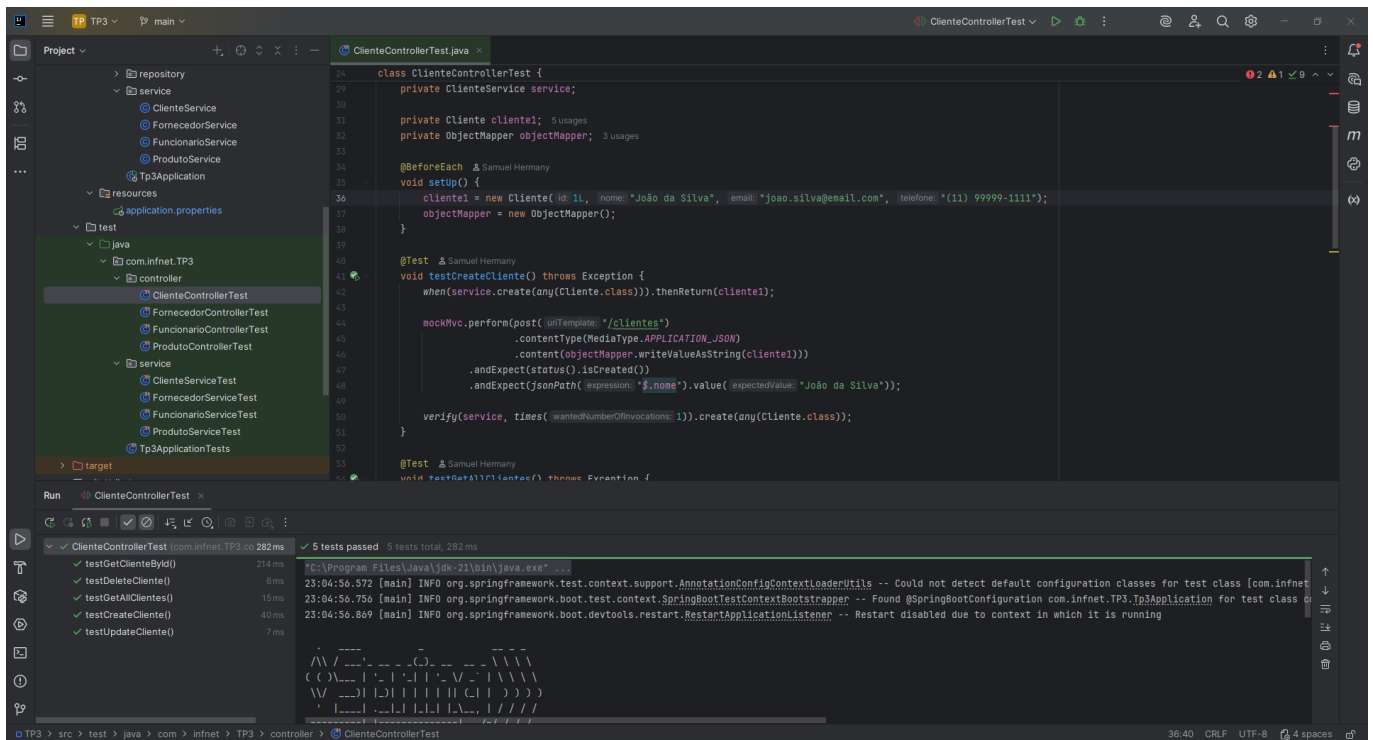
4. Exemplo de Persistência

```
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Setter
@EqualsAndHashCode(of = "id")
@Table(name = "tb_cliente")
@Entity
public class Cliente implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String email;
    private String telefone;
}
```

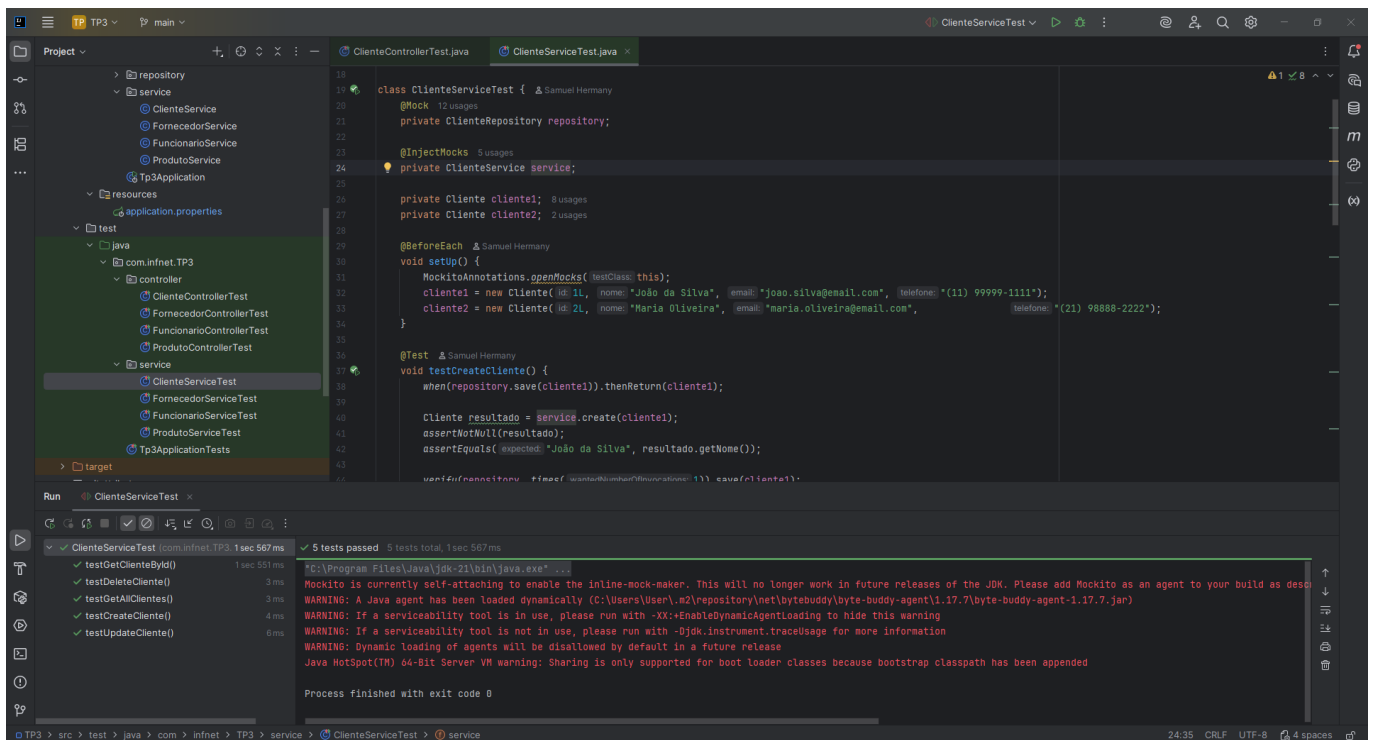
4. DESENVOLVIMENTO DE TESTES NO SERVIÇOS REST

1. Cliente

- ClienteControllerTest

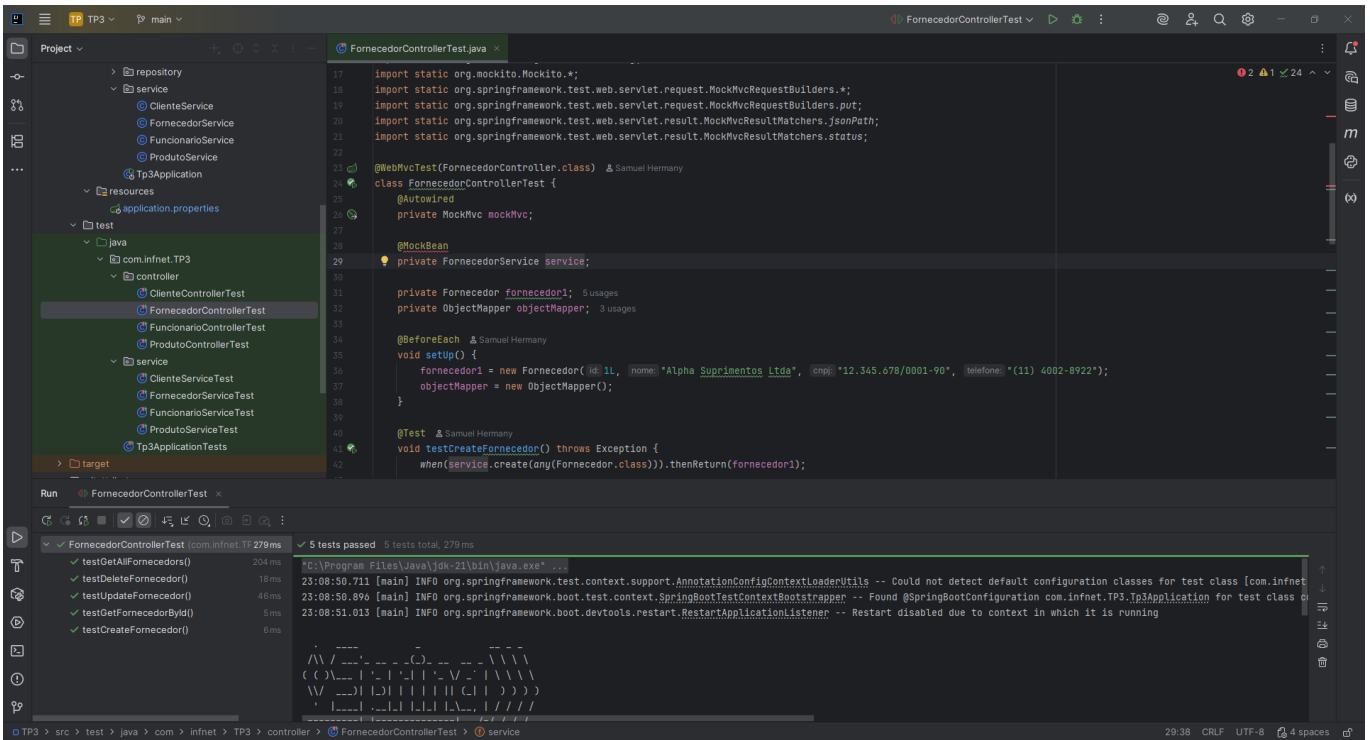


- ClienteServiceTest

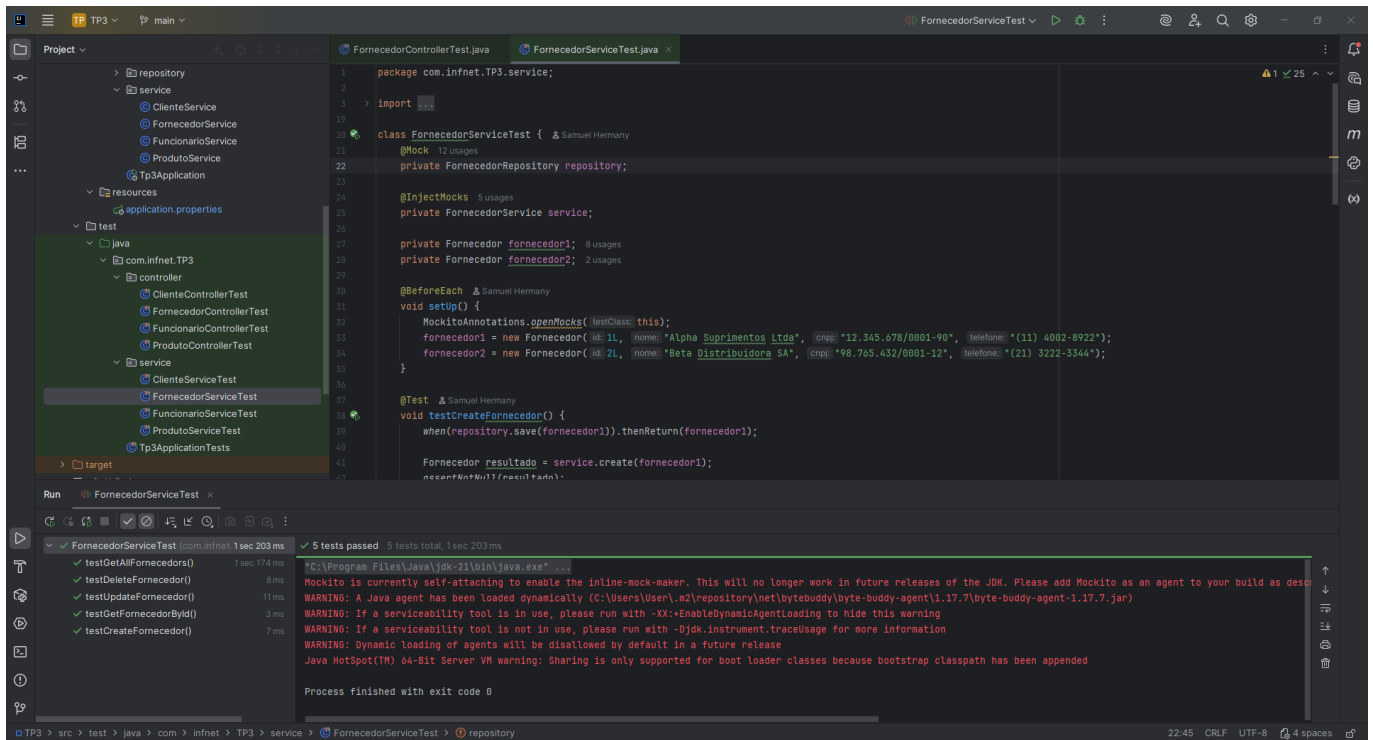


2. Fornecedor

- FornecedorControllerTest

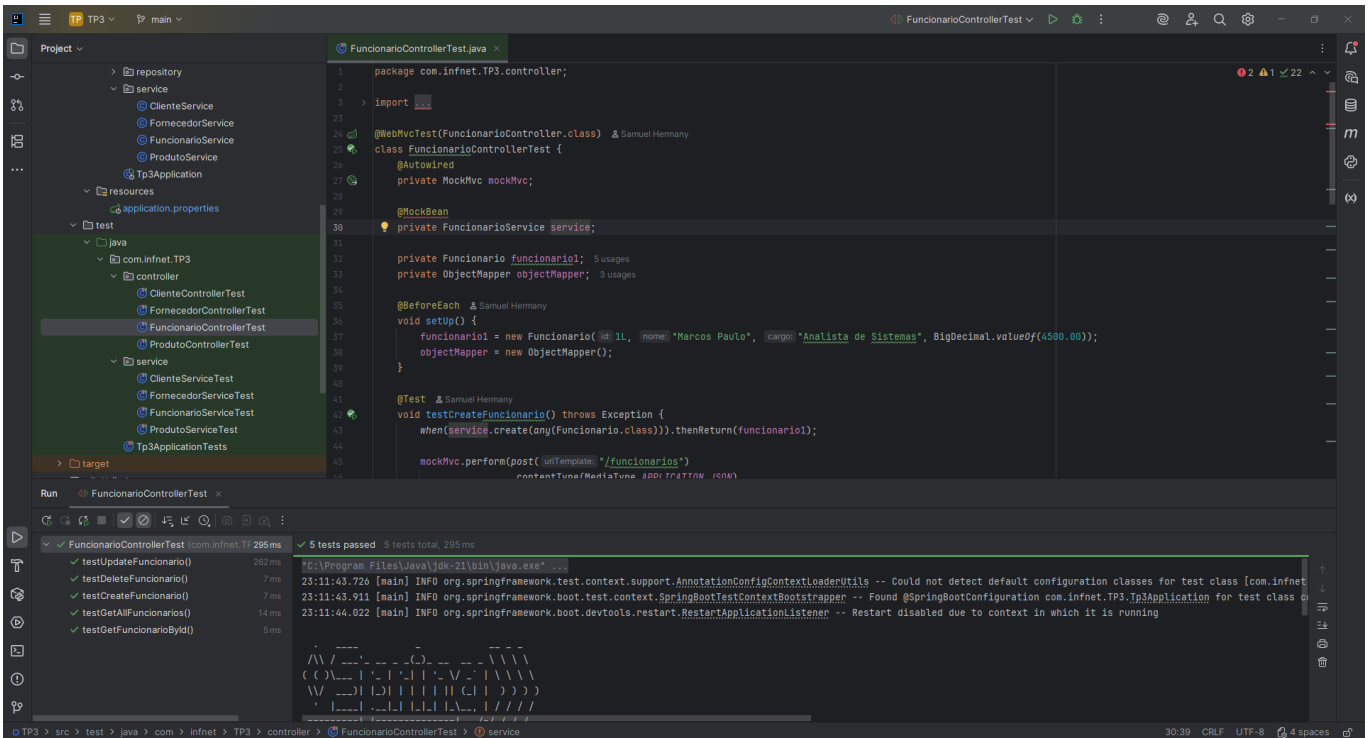


- FornecedorServiceTest

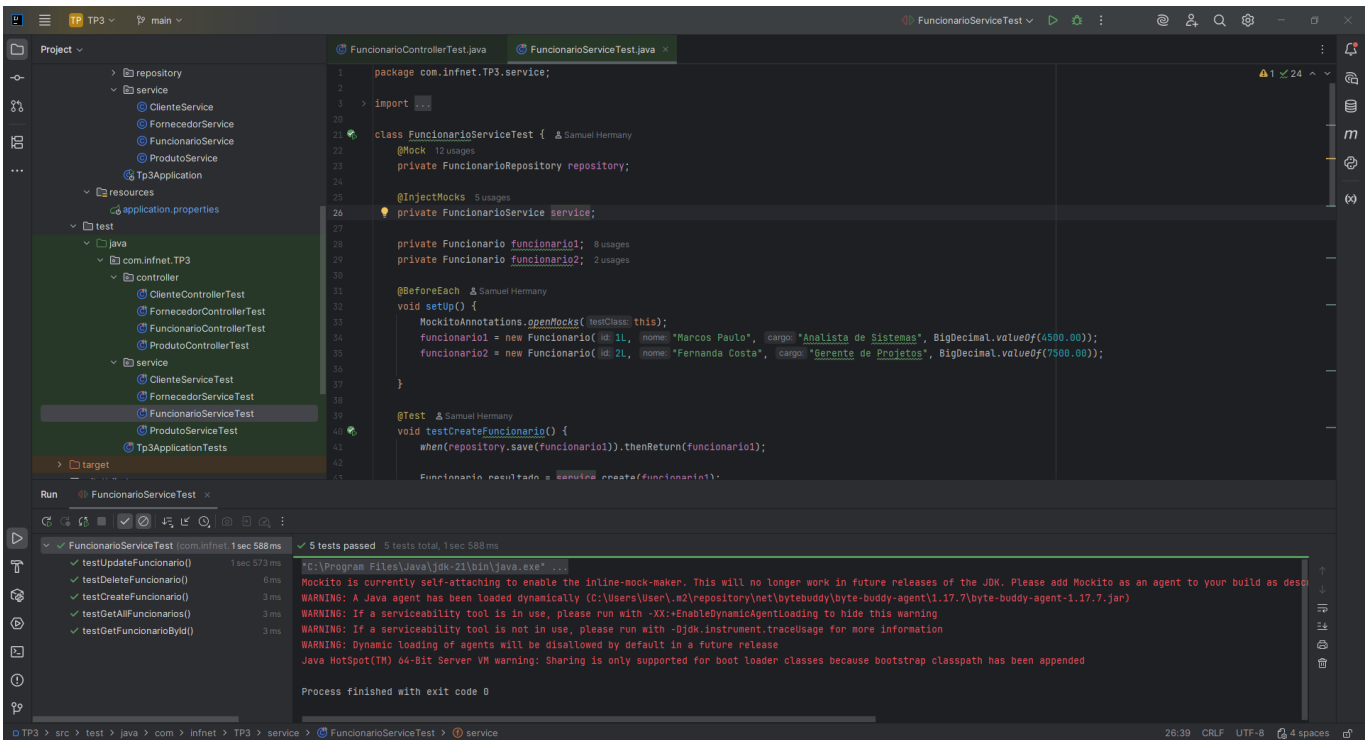


3. Funcionario

- FuncionarioControllerTest

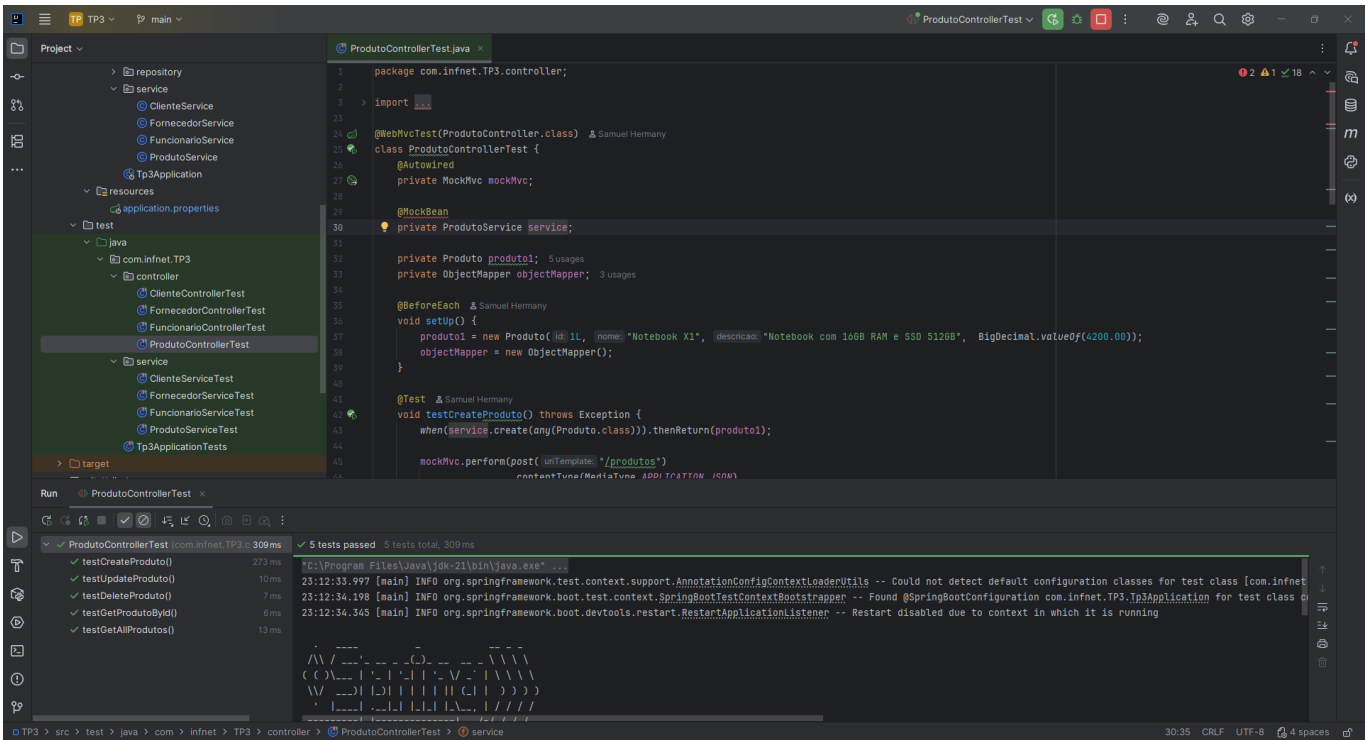


- FuncionarioServiceTest



4. Produto

- ProdutoControllerTest



- ProdutoServiceTest

